OVERVIEW

Computer can only understand one language, **machine language**. Machine language is very difficult to learn because it consists only of the binary numbers 0 and 1.

So we use languages that we know as well as there are some ways of converting this to machine language. It is very hard to convert natural languages, so special programming languages have been developed similar to natural languages (specially in English). Therefore these are easy to learn. These programming languages are used to develop computer programs that give the computer a detailed set of instructions to execute.

It is very tempting to start entering your program into the computer as soon as you have some idea of how to write it. Instead, you need to analyse the problem and its solution before you write any program instructions.

It has been clearly demonstrated that time spent defining the problem to be solved and then designing the solution leads to more efficient programming and fewer errors in the program.

PROGRAM DEVELOPMENT CYCLE

The task of creating a program that meets the user's requirements and is adaptable is a complex and difficult task unless it is organised. This is done using a cycle of well-defined steps to create simpler parts to reduce the complexity.

A number of stages can be identified in the program development process. These stages are referred to as the **Program Development Cycle** to emphasise the point that it may be necessary to return to an earlier stage if errors or ambiguities are detected.

Six stages of program development can be identified:

1. Problem definition.
2. Development of a solution.
3. Testing of the solution.
4. Coding of the solution.
5. Testing the program.
6. Maintenance and documenting the program.

1. PROBLEM DEFINITION

This stage is the most important because all subsequent stages are dependent on it. Problem definition involves analysing the problem until a clear understanding is reached. Without a clear understanding of the problem, time and effort spent on later stages can be wasted. It may be necessary to have a discussion with the person who proposed the problem to clarify issues.

In a business environment the programming problem is generally presented as a **program specification**, which sets out

i) Input,
ii) Processing
iii) Output requirements and
iv) Other requirements of the program.

The program specification should also include known results for given inputs to the program. Sometimes these known results are termed as test cases. These known results can be used as a guide to the successful solution of the problem.

An **IPO** chart is a tool that can be used to help document the problem definition.

> An IPO chart is a table that specifies the input, processing and output requirements of the program.

IPO charts are sometimes called **defining diagram**.

An accessory to the IPO chart is a list of names for each of the input and output data storage. These names are easier to use than a full text description of an item and can lead to the removal of ambiguity. This list is referred to as the **Initial Data Dictionary**. Other data items, which are neither input nor output, are added to this list as the processing steps are developed. The complete list of data items used in a program is referred to as the **Data Dictionary** for the program.

2. DEVELOPMENT OF A SOLUTION

The technique of **top-down design** in the development of a solution involves breaking the problem down into a number of tasks to establish an outline of the solution, and then breaking these tasks down into sub-tasks until the solution of each task is manageable.

The solution outline can be represented by a **hierarchy or structure chart**. Hierarchy charts show the relationships between sections in a program.

The set of logical steps that describe the actions to be carried out in each task to solve the overall problem is called an **algorithm**.

There are many different techniques used to represent an algorithm. Algorithms can be **diagrammatic** or **descriptive**.

Diagrammatic algorithms include:
1. Flowcharts.
2. Nassi-Shneiderman diagrams.
3. Action diagrams.

Descriptive algorithms include:
1. Pseudocode.
2. Point form.
3. Structured English.

3. TESTING OF THE SOLUTION

It is vital to check that the algorithm actually solves the problem and produces the expected results for given inputs. In addition to the known results that may have been included in the program specification, other test data should be designed to test the logic of the solution.

A technique for testing the algorithm with test data is the **desk check**. This is a step-by-step walk-through of the logic of the algorithm to see how the input is transformed into output. This is a paper (or whiteboard) exercise that is carried out before the program code is entered on the computer. If the results from the desk check do not match the expected results it is necessary to return to stage 2 to modify the algorithm. The modified algorithm is then retested and modified until the expected results are obtained for the test data. Since the conclusions derived from a desk check are only as good as the test data, it is necessary to design comprehensive test data, which should test all conceivable conditions.

4. CODING THE PROGRAM

The commands of a programming language are used to transform the algorithm into a computer program after the logic of the program has been planned. Any programming language can be used to code the algorithm, however a particular language may be more suitable than others depending on the type of problem.

During the coding of the program it is good practice to include comments that explain the logic of the processing steps. These comments will make it easier to understand the program at a later date and make the maintenance of the program easier.

A compiler or interpreter is used to convert the program code to machine language that the computer can understand. When the compiler detects an error in the program, an error message is displayed.

There are two basic categories of error messages:

1. **Syntax** and
2. **Run-time**.

The compiler detects and displays syntax errors as it attempts to translate the program. If the program contains a syntax error, it cannot be translated and the program cannot be executed.

Syntax errors occur if the rules of the programming language (the syntax) are not adhered to. This may occur when reserved words (words that have special meaning to that programming language) are misused, misspelled or punctuated incorrectly. These syntax errors must be corrected before the program can be run. The program code is stored in a text file, which can be printed from the program editor, a text editor, or a word processing package. The **program listing** is the documentation for this stage.

Run-time errors are detected and displayed during execution of the program. They occur when the program instructs the computer to perform an illegal operation, such as subscript out of range, dividing a number by zero, type mismatches or manipulating undefined or invalid data. When a run-time error occurs, the computer stops program execution and displays an error message.

5. TESTING THE PROGRAM

This step involves running the program with the same test data used to test the algorithm. If the output does not match the expected results it indicates that errors have been made in converting the algorithm into program code. Debugging the program is the process that identifies these errors.

A number of techniques can be used to check the progress of the processing steps. Many program editors have a **debugger**, which can be used for this purpose. When an error is identified, the program is modified and then tested again. This cycle is repeated until the program gives the required output.

6. MAINTENANCE OF THE PROGRAM AND DOCUMENTATION

Documentation consists of the written materials produced at each stage of the program development process. It is a cumulative process that starts with the initial statement of the problem, adds the details of the solution design, then the coding and details of the testing.

One document that is developed over several of the stages in the program development cycle is the **Data Dictionary**. This is an alphabetic list of the names of all the data items used by the program. It includes a description of the data item and the type of data. The initial data dictionary from the problem definition stage is added to during algorithm development and during coding as extra data items are used in the steps to convert the input to output.

The data dictionary is provided both as written documentation and as a series of comments in the first part of the program. The written document is used as a reference for data items in the algorithm; the data dictionary in the program provides the programmer, and other programmers, with a reference section to ensure consistency in the use of data within the program.

Once the program has been thoroughly tested, the documentation should be reviewed to ensure consistency between the materials produced at each stage.

Documentation for a program consists of:

1. Program specification.
2. IPO chart.
3. Hierarchy or structure chart.
4. Algorithm in pseudocode or NS diagrams or flowcharts.
5. Test data and desk check.
6. Program listing.
7. Results of testing the program.

It is likely that the program will be changed at some time after it has been successfully designed and tested. Errors may occur or modifications may be requested. As changes are made they should be documented and the materials from each stage modified to reflect those changes.

PROBLEM DEFINITION

A program specification is a formal way of documenting what a computer program is to do. The format of a program specification should contain the following:

1. A general **DESCRIPTION** of the program i.e. what it is required to do.

2. Program **INPUTS** - a list of data items which are to be provided as inputs to the program and where these inputs are coming from e.g. keyboard, file etc.

3. The **PROCESSING** requirements - a description of how the program inputs are to be processed so that the required program outputs are produced.

4. Program **OUTPUTS** - a list of those results which are to be produced by the program and where the outputs are going e.g. screen, printer, file etc.

5. The **OVERALL TASKS**, i.e. the broad solution steps (the highest level of top-down design) or an overall view of the tasks the program will have to perform.

The purposes of the program specification are:

1. It helps to clarify what the program is required to do.

2. Any problem ambiguities are removed.

3. It gives the programmer an excellent starting point for thinking about the details of the algorithm.

Below is an example of a program specification.

Example 1.1

DESCRIPTION

The Computer Disk Supply Company has made a request for a program to handle the ordering of their floppy disks. The diskettes are supplied in boxes of 10 and prices vary according to size and density of the diskettes. The program is to ask for the details of an order then give the cost of the order entered. After an order is entered the user is to be given the chance to continue to enter another order (by pressing the enter key) or to terminate the input of orders by entering the word END. Once the user enters END, some statistics on all orders entered are to be provided.

VARIABLE	DESCRIPTION	SOURCE
NAME	The name of the person/company ordering.	From keyboard.

PROCESSING

1. Validate each of the inputs as they are entered. If an invalid value is entered an error message should be printed and a new value requested until a valid value is entered.
2. Calculate the order cost as follows:
    - 3.5" diskettes cost $20 per box for double density.
    - 5.25" diskettes cost $10 per box for double density.
    - 8" diskettes cost $15 per box for double density.
    - For 3.5" and 5.25" diskettes add $3 per box for high density.
3. Statistics includes:
    - The number of orders processed.
    - The number of boxes of each size of diskette sold (all densities combined).
    - The cost of all orders.

OUTPUT

1. For each order print the details of the order (person/company name, size, density, number of boxes and order cost) to the screen.
2. After the orders have been entered i.e. after the user enters END to finish the session the following are displayed on the screen:
    - The total number of orders made.
    - The total number of boxes of each size (double and high combined).
    - The total cost of all orders.

OVERALL TASKS

1. Initialisations
2. Input and validate order details or END
3. Calculate order cost
4. Update statistics
5. Print details of order
6. Print statistics for all orders entered

Exercise 1

In each of the following write a program specification.

1. **POST0**: A program outputs to the screen the postage charge for a parcel when the weight of the parcel (in Kg) is entered from the keyboard. The charge is set at $1.50 per kg.

2. **BANK0**: A loan officer requires a program to calculate the monthly repayment M on a customer loan by entering from the keyboard the principal P, at an annual interest rate of R% p.a., over a period of T years where M=P*(1+R*T/100)/(12*T). All inputs and the result is output to the screen.

3. **ANIM0**: The consultation fee for a vet is obtained by multiplying the number of hours by the hourly rate of $25. A program accepts the number of hours (rounded up) from the keyboard and outputs the consultation fee to the screen.

4. **TEMP0**: Convert a temperature entered from the keyboard in degrees Fahrenheit to degrees Celsius where degC=(degF-32)/1.8 and output the result to the screen.

5. **RECT0**: A program accepts the height and width of a rectangle from the keyboard, calculates the perimeter and area of the rectangle and outputs the results to the screen.

6. **DISC0**: A program accepts the marked value of an item in $ and cents, and the discount rate as a % from the keyboard and returns the discounted price to the screen.

7. **TIME0**: A program allows a person to enter values for the number of days, hours, minutes and seconds and outputs the total number of seconds to the screen.

One means of coming to understand a programming problem is to identify and state the following details:

1. The data provided by the user (INPUT).
2. The processes involved in solving the problem (PROCESSING).
3. The information that is provided to the user (OUTPUT).

A program will usually require some form of **input**, be it information to process or the selection of a processing option from a menu. You usually cannot do any processing until you have completed the input of information to process.

The program will often then **process** some of the input and convert it to some form of information based on the data supplied. The processing steps should be described only as actions at this stage. The details about how each step will be carried out are developed as part of the algorithm design.

The last section of a program often involves **output**. This can be in the form of a report displayed on the screen or output to a printer, or it could be information written to some sort of storage device, such as disk or tape.

The IPO chart should be able to be read and understood by a non-programmer. It may provide the basis for discussion with the person who proposed the program if ambiguous details need to be clarified. Thus, the entries in the IPO chart should be in plain English.

Example 1.2

Problem: A program is required that will allow a user to enter a temperature in degrees Celsius and will display the temperature in degrees Fahrenheit.

Problem definition:

An analysis of the problem indicates a single input and a single output. The input has to be converted to the output.

IPO CHART

INPUT	PROCESSING	OUTPUT
Temperature in degrees Celsius	1. Enter the temperature in degrees Celsius	Temperature in degrees Fahrenheit
	2. Convert the temperature in degrees Celsius to the temperature in degrees Fahrenheit	
	3. Display the temperature in degrees Fahrenheit	

Note: The precise details of converting a temperature in degrees Celsius to a temperature in degrees Fahrenheit are not required at this stage. It is sufficient to describe **what** is to be done, not **how** it will be done.

The IPO chart should now be analysed and modified until the following questions are satisfied:

1. Are all the inputs required for the processing steps specified?
2. Are all of the required outputs specified?
3. Do the processing steps transform input to output?

Exercise 2

Problem 1:

Using an IPO Chart, determine how an employee's gross pay and net pay is determined. The employees would need to be prompted to enter their hours worked and the hourly rate. The result would be displayed on the screen.

where

gross pay = hours worked x hourly rate
tax amount = 10% of gross pay
net pay = gross - tax amount

Problem 2:

A program is required to perform arithmetic calculations on two integers. The program should prompt for the input of the two integer numbers and display the following on the screen:

a. The sum of the two numbers.
b. The difference of the two numbers.
c. The product of the two numbers.
d. The quotient of the two numbers.

Develop an IPO Chart.